7. SYSTEM ARCHITECTURE AND WORKFLOW

7.1 Architectural Overview and Multi-Tier Decomposition

The IntelliTwin framework is structured around a decoupled, multi-tier cyber-physical research architecture designed to resolve the pervasive gap between data-driven Remaining Useful Life (RUL) estimation, finite-sample distribution-free uncertainty calibration, and sequential operational maintenance decision-making [1]–[3]. In industrial rotating machinery, such as commercial turbofan aircraft engines, standard point predictions optimize symmetric statistical metrics that fail to reflect asymmetric downtime losses (where unscheduled in-service failure penalty significantly exceeds scheduled preventive overhaul cost, $C_{\text{FAIL}} \gg C_{\text{PM}}$) [1], [5]. Concurrently, digital twin implementations in manufacturing literature frequently prioritize high-fidelity 3D geometric visualizations or centralized SCADA dashboards without mathematically formalizing how predictive uncertainty should govern discrete intervention decisions [2], [13], [14].

To bridge these disjoint paradigms, IntelliTwin decomposes asset monitoring, prognostic estimation, uncertainty calibration, and maintenance simulation into five modular, decoupled architectural tiers:

1. **Tier 1: Data Ingestion and Anti-Leakage Preprocessing Tier**: Ingests multi-sensor run-to-failure streams from the NASA C-MAPSS corpus, enforces immutable engine-level disjoint split isolation, performs invariant sensor pruning, fits training-only normalization parameters, clusters operational settings, and constructs causal sliding lookback windows [1].
2. **Tier 2: Prognostic Base Inference Tier**: Houses complementary shallow gradient-boosted decision trees (`HistGradientBoostingRegressor`) and deep recurrent neural networks (PyTorch `LSTM`), generating continuous point predictions of Remaining Useful Life [3], [4].
3. **Tier 3: Hierarchical Conformal Calibration Tier**: Computes asymmetric nonconformity scores, evaluates exact engine-weighted quantiles under cluster exchangeability, stratifies calibration bounds across operational flight regimes, and inverts scores into finite-sample prediction intervals $[L_i(t), U_i(t)]$ targeting 90% nominal coverage [7], [9].
4. **Tier 4: Sequential Decision and Maintenance Simulation Tier**: Evaluates dynamic cycle-by-cycle stopping rules $\tau_i = \inf\{t \geq W : L_i(t) \leq \theta\}$ against conservative lower conformal bounds, tracks economic lifecycle expenditures in

Standardized Cost Units (SCU), executes matched-conservatism attribution controls, and conducts 10,000 cluster bootstrap hypothesis tests [5], [8], [12].

5. **Tier 5: Digital Twin Synchronization and Presentation Tier**: Formulates a real-time trace-driven state replication engine (`TwinReplayEngine`) adhering to ISO 23247 principles [14], guarantees strict numerical verification parity (atol $\leq 10^{-4}$) against offline batch execution, and exposes streaming REST endpoints via FastAPI to an interactive HTML5/JavaScript operator dashboard [2], [13].

Physical Integration and Research Demarcation

In alignment with rigorous academic reporting standards, the physical integration boundary of the IntelliTwin system is explicitly demarcated:

- **Offline Trace-Driven Simulation**: The architecture operates on historical run-to-failure telemetry from the NASA C-MAPSS simulator [1]. No live turbofan engine is physically connected to the host system, and no hardware IoT sensor bus (e.g., MQTT, OPC UA) is claimed.
- **Advisory Decision Support**: Maintenance orders dispatched by the simulation tier represent advisory recommendations for maintenance engineers; closed-loop physical actuation (e.g., automatic engine shutdown or thrust throttling) is strictly out of scope.
- **Strict Anti-Leakage Boundary**: Held-out test trajectories are isolated from model training, hyperparameter grid search, and conformal score calibration. True failure times and oracle RUL values are revealed exclusively to offline evaluation routines for post-hoc scoring.

7.2 Data Ingestion, Normalization, and Causal Windowing Pipeline

The data pipeline ingests raw continuous multi-sensor records and transforms them into standardized tensor and vector representations while strictly eliminating temporal and inter-engine data leakage.

C-MAPSS Data Schema and Invariant Channel Pruning

The raw C-MAPSS dataset provides 26 columns per cycle: engine unit identifier, cycle index, 3 operational flight condition settings, and 21 thermo-mechanical degradation sensors [1]. Across multi-regime datasets (FD002, FD004), a systematic variance audit identified six invariant or uninformative channels exhibiting zero variance or uninformative responses across all regimes:

$$S_{dropped} = \{ s_1 \text{ (T2)}, s_5 \text{ (P2)}, s_6 \text{ (P15)}, s_{10} \text{ (epr)}, s_{16} \text{ (farB)}, s_{19} \text{ (Nf_dmd)} \}$$

Excluding these uninformative channels eliminates matrix rank deficiency and numerical instability. The retained schema comprises $d = 17$ informative channels: 3 operating settings (z_1, z_2, z_3) and 14 dynamic thermo-mechanical sensors (temperatures, pressures, core/fan speeds, fuel ratios, and coolant bleeds).

Strict Engine-Level Split Partitioning

To prevent inter-engine correlation leakage, partitioning is strictly enforced at the engine unit level. For each benchmark dataset, entire run-to-failure engine trajectories belong exclusively to one split:

- **FD001 (Single-Regime Control)**: 100 total engines split into 50 Training, 20 Calibration, 10 Validation, and 20 Test engines.
- **FD002 (Multi-Regime Primary)**: 260 total engines split into 130 Training, 52 Calibration, 26 Validation, and 52 Test engines.
- **FD004 (Multi-Regime / Multi-Fault Robustness)**: 249 total engines split into 124 Training, 50 Calibration, 25 Validation, and 50 Test engines.

Training-Only Normalization and Regime Clustering

Standardization parameters (mean $\boldsymbol{\mu}_{\text{train}}$ and population standard deviation $\boldsymbol{\sigma}_{\text{train}}$, computed with `ddof=0`) are derived solely from the designated Training engines:

$$\tilde{\boldsymbol{x}}_i(t) = (\boldsymbol{x}_i(t) - \boldsymbol{\mu}_{train}) / (\boldsymbol{\sigma}_{train} + \varepsilon)$$

where $\varepsilon = 10^{-8}$ prevents division by zero. For multi-regime sub-corpora (FD002, FD004), operating settings (z_1, z_2, z_3) are standardized and clustered into $K = 6$ operational regimes (C_1 to C_6) via K-means clustering ($K = 6$, `random_state=2026`, `n_init=10`) fitted exclusively on training data, preserving deterministic centroid ordering.

Causal Sliding-Window Formulation

To preserve temporal causality, feature representations are constructed using a fixed sliding lookback window of length $W = 30$ cycles with stride 1:

$$\boldsymbol{X}_i(t) = [\, \tilde{\boldsymbol{x}}_i(t - W + 1), \tilde{\boldsymbol{x}}_i(t - W + 2), \ldots, \tilde{\boldsymbol{x}}_i(t) \,]^T \in \mathbb{R}^{30 \times 17}$$

For early operational cycles ($1 \le t < W = 30$), the causal buffer is filling; zero padding is strictly forbidden, and prognostic inference is deferred until cycle $t = 30$. Dual feature representations are constructed from this window:

- **Sequential Tensor**: A 2D matrix $\mathbf{X}_i(t) \in \mathbb{R}^{30 \times 17}$ preserving temporal ordering for the recurrent LSTM network.
- **Flattened Vector**: A 1D vector $\mathbf{v}_i(t) = \text{vec}(\mathbf{X}_i(t)) \in \mathbb{R}^{510}$ flattened in C-order (row-major) for gradient-boosted decision trees.

7.3 Prognostic Base Inference Tier

The prognostic inference tier transforms causal window representations into continuous point estimates of Remaining Useful Life $\hat{y}_i(t)$.

Dual Target Conventions

During base model training, Remaining Useful Life targets are clamped at an upper threshold $R_{\max} = 125$ cycles:

$$y_{train,i}(t) = min(R_{max}, T_i - t)$$

reflecting the empirical physical reality that initial operational wear does not exhibit measurable degradation signatures during early component life [1]. In contrast, during held-out test evaluation, the true uncapped linear degradation target $y_{\text{eval},i}(t) = T_i - t$ is preserved to assess genuine lifetime prognostic accuracy.

Model Architecture Workflows

Two diverse, complementary model families are trained and frozen under Architecture Decision Record ADR-018:

- **HistGradientBoostingRegressor (GBR)**: Ingests flattened 510-dimensional vectors $\mathbf{v}_i(t)$. Optimizes the asymmetric pinball loss function $L_\rho(y, \hat{y}) = \max(\rho(y - \hat{y}), (1 - \rho)(\hat{y} - y))$ across quantiles $\rho \in \{0.05, 0.50, 0.95\}$ with histogram-binned split finding (max bins = 255, learning rate = 0.05, max leaf nodes = 31, L2 reg = 1.0).
- **PyTorch Deep Bidirectional LSTM**: Ingests sequential tensors $\mathbf{X}_i(t) \in \mathbb{R}^{30 \times 17}$. Features two stacked bidirectional LSTM recurrent layers ($h = 64$ hidden units per direction, 128 concatenated), variational recurrent dropout ($p = 0.20$), and a LeakyReLU projection head. Optimized using AdamW ($\eta = 10^{-3}$, weight decay $= 10^{-4}$) under Mean Squared Error loss with cosine annealing scheduling.

Point predictions are non-negatively clipped: $\hat{p}_i(t) = \max(0, \hat{y}_i(t))$. A total of 30 model checkpoints (3 datasets × 2 architectures × 5 seeds [2026..2030]) are registered and frozen.

7.4 Hierarchical Conformal Calibration and Interval Inversion Workflow

The conformal calibration tier converts raw point predictions into statistically guaranteed, finite-sample prediction intervals $C_{1-\alpha}(\mathbf{X}_i(t)) = [L_i(t),\ U_i(t)]$ targeting nominal 90% coverage ($1 - \alpha = 0.90$) [4], [7], [9].

Asymmetric Decision-Aware Nonconformity Scoring

To penalize hazardous overestimation ($\hat{y} > y$, risking in-service failure) more heavily than premature underestimation ($\hat{y} < y$, resulting in early inspection), nonconformity scores incorporate state-dependent dynamic scaling [7]:

$$\sigma(\hat{p}) = (1 + \hat{p} / 125.0)^{\gamma}, \qquad z(\hat{p}) = \exp(-\hat{p} / 20.0)$$
$$a(\hat{p}) = \sigma(\hat{p}) \cdot \exp(\beta \cdot z(\hat{p})), \qquad b(\hat{p}) = \sigma(\hat{p}) \cdot \exp(-\beta \cdot z(\hat{p}))$$
$$s_i(t) = \max\left((\hat{p}_i(t) - y_i(t)) / a(\hat{p}_i(t)),\ \ (y_i(t) - \hat{p}_i(t)) / b(\hat{p}_i(t)),\ \ 0 \right)$$

with candidate parameters $\gamma = 2$ and $\beta = -\ln 2 \approx -0.693147$, optimized via grid search on the Validation split.

Exact Engine-Weighted Quantile Solver

To resolve the clustered temporal autocorrelation of repeated observations within engines, each observation (i, t) from calibration engine $i \in \{1, \ldots, m\}$ is assigned weight $w_{i,t} = 1 / ((m + 1) \cdot n_i)$. Total finite probability mass equals $m / (m + 1)$, with the remaining $1 / (m + 1)$ assigned to $+\infty$ [9]:

$$F_{dev}(q) = (1 / (m + 1)) \cdot \sum_{i=1..m} \left[(1 / n_i) \cdot \sum_{t=1..n_i} \mathbb{I}(s_{i,t} \le q) \right]$$

The conformal quantile $\hat{q}_{1-\alpha}$ is the smallest observed calibration score satisfying $F_{\text{dev}}(\hat{q}_{1-\alpha}) \ge 1 - \alpha$. When calibrating within specific operational flight regimes $k \in \{1, \ldots, K\}$, condition-adaptive thresholds $\hat{q}_{1-\alpha}^{(k)}$ are computed; if a regime cluster contains fewer than $m_g \ge 10$ calibration engines, fallback routing to the global threshold $\hat{q}_{1-\alpha}^{(\text{global})}$ is automatically triggered.

Closed-Form Prediction Interval Inversion

Given the calibrated threshold $\hat{q}_{1-\alpha}$, the finite-sample prediction interval is evaluated via closed-form inversion:

$$L_i(t) = \max(0,\ \hat{p}_i(t) - \hat{q}_{1-\alpha} \cdot a(\hat{p}_i(t))), \qquad U_i(t) = \hat{p}_i(t) + \hat{q}_{1-\alpha} \cdot b(\hat{p}_i(t))$$

where $L_i(t)$ represents the conservative lower bound on Remaining Useful Life that directly drives downstream maintenance policies.

7.5 Discrete-Event Sequential Maintenance Utility Simulation Workflow

The simulation tier models asset lifecycle progression cycle-by-cycle using a discrete-event simulator (`src/intellitwin/simulator/engine.py`) [5], [8].

Sequential Stopping Policy

At each operational cycle $t \geq 30$, the simulator evaluates the lower conformal bound $L_i(t)$. When the lower bound breaches an operational threshold θ (with default $\theta = 1.0$ cycle), a preventive maintenance order is dispatched immediately:

$$\tau_i = inf\{\, t \geq W : L_i(t) \leq 1.0 \,\}$$

If $\tau_i < T_i$, planned maintenance is successfully executed at cycle τ_i, terminating the trajectory without in-service incident. If the engine reaches its terminal cycle before maintenance is triggered ($\tau_i \geq T_i$), an unscheduled catastrophic failure occurs at cycle T_i.

Standardized Cost Units (SCU) Evaluation

Total simulated lifecycle cost for engine i is calculated under the primary reference scenario:

$$Cost_i(\tau_i) = C_{PM} + C_{WASTE} \cdot (T_i - \tau_i) \quad \text{if } \tau_i < T_i \text{ (Planned Maintenance)}$$
$$Cost_i(\tau_i) = C_{FAIL} \quad \text{if } \tau_i \geq T_i \text{ (Unscheduled In-Service Failure)}$$

with economic parameters: planned overhaul $C_{\mathrm{PM}} = 1.0$ SCU, in-service failure penalty $C_{\mathrm{FAIL}} = 5.0$ SCU, and wasted component life penalty $C_{\mathrm{WASTE}} = 0.20$ SCU per cycle.

Controlled Attribution Protocol

To verify that cost reductions stem from genuine uncertainty calibration rather than heuristic over-conservatism, outcomes are evaluated against three pre-registered matched-conservatism control families:

1. **Uniformly Widened Intervals (δ-Widened)**: $\hat{q}' = \hat{q} + \delta$.
2. **Point Safety Margins (Δ-Margin)**: $\hat{p}'(t) = \hat{p}(t) - \Delta$.
3. **Earlier Scheduled Intervention (θ-Offset)**: $\theta' = \theta + \tau_{\mathrm{advance}}$.

Statistical significance is established via 10,000 engine-level cluster bootstrap draws under Bonferroni family-wise error rate control (α = 0.0125) and Two One-Sided Tests (TOST) non-harm equivalence bounds [12].

7.6 Digital Twin State Synchronization and Replay Engine

To translate scientific models into an interactive operational prototype, IntelliTwin implements a real-time trace-driven Digital Twin synchronization engine (`TwinReplayEngine`) in `src/intellitwin/twin/replay.py` [2], [13], [14].

Thread-Safe Replay State Machine

The replay engine manages asset state transitions through an explicit lifecycle state machine protected by mutual exclusion locks:

- **WARMING_UP (Cycles 1 … 29)**: Sensor frames are buffered into the causal history. Inference is deferred; no prediction bounds or maintenance actions are emitted.
- **ACTIVE (Cycles 30 … min(τ_i, $T_i - 1$))**: The engine maintains the active 30 × 17 sliding window, applies training Z-scores, computes dual GBR/LSTM forecasts, derives conformal bounds [$L_i(t)$, $U_i(t)$], calculates dynamic Health Index $\mathrm{HI}_i(t) = \min(1.0, \hat{p}_i(t) / 125)$, and evaluates maintenance policy rules.
- **MAINTAINED (Cycle τ_i)**: Triggered when $L_i(t) \leq 1.0$. The session locks, records a planned overhaul event, and halts further telemetry advancement.
- **FAILED (Cycle T_i)**: Reached if cycle t attains total failure cycle T_i without intervention. Records catastrophic failure and terminates the session.

Numerical Parity Verification

To guarantee that real-time replay does not introduce numerical artifacts or windowing drift, the replay engine is audited against offline batch execution:

$$\max_t \left| \hat{p}_{stream}(t) - \hat{p}_{batch}(t) \right| \leq 10^{-4}, \qquad \max_t \left| L_{stream}(t) - L_{batch}(t) \right| \leq 10^{-4}$$

Parity is verified across all benchmark datasets via 37 automated pytest modules, establishing that the interactive operator dashboard displays mathematically identical outputs to the scientific evaluation pipeline.

REST API Architecture and Operator Dashboard

The replay engine is exposed via a high-performance FastAPI service (`src/intellitwin/twin/api.py`):

- `GET /api/health`: System health check, runtime metadata, and frozen model registry status.
- `GET /api/catalog`: Available benchmark datasets (FD001, FD002, FD004), model families, and test engines.
- `POST /api/sessions`: Spawns a stateful replay session for a designated engine unit.
- `POST /api/sessions/{id}/advance`: Consumes exactly one cycle observation, returning updated telemetry, predictions, bounds, and maintenance status.
- `POST /api/sessions/{id}/reset`: Resets the replay session to cycle 0.
- `GET /api/sessions/{id}/trace.csv`: Exports complete cycle-by-cycle operational traces for auditing.

The frontend interface (`apps/twin_dashboard/`) renders an interactive HTML5/JavaScript cockpit featuring dynamic sensor time-series charts, RUL prediction intervals with confidence bands, gauge dials for Health Index, and visual alert indicators (`NOMINAL`, `ADVISORY`, `CRITICAL_INTERVENTION`).

7.7 Architectural Schematics and Workflow Diagrams

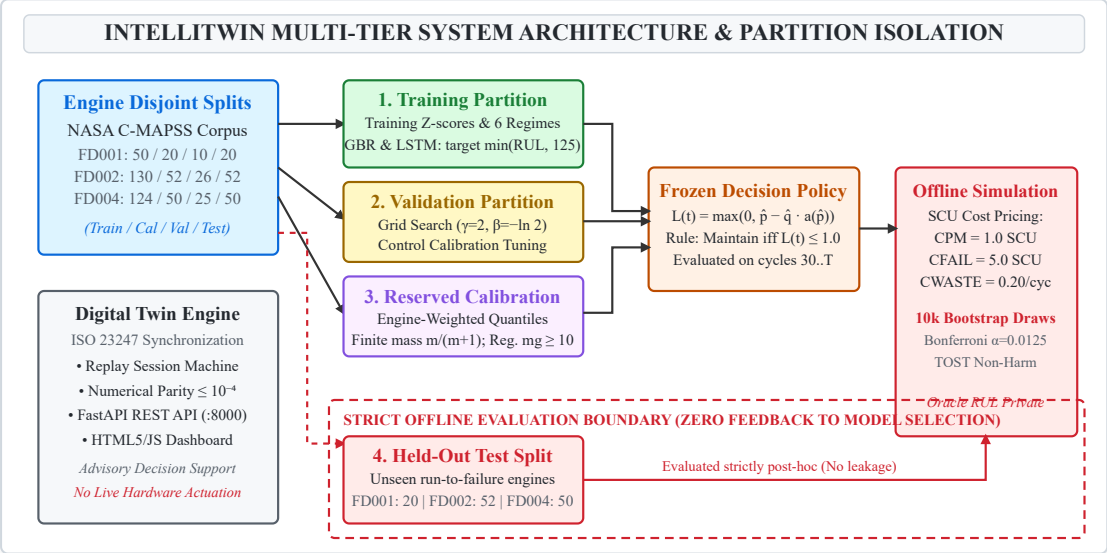


Figure 7.1: Multi-Tier System Architecture and Information Flow. Telemetry is partitioned by physical engine unit across disjoint splits. Training parameters and conformal calibration bounds flow strictly forward into the frozen decision policy. Held-out test evaluation is isolated behind a strict offline boundary with zero feedback to model selection or parameter tuning.

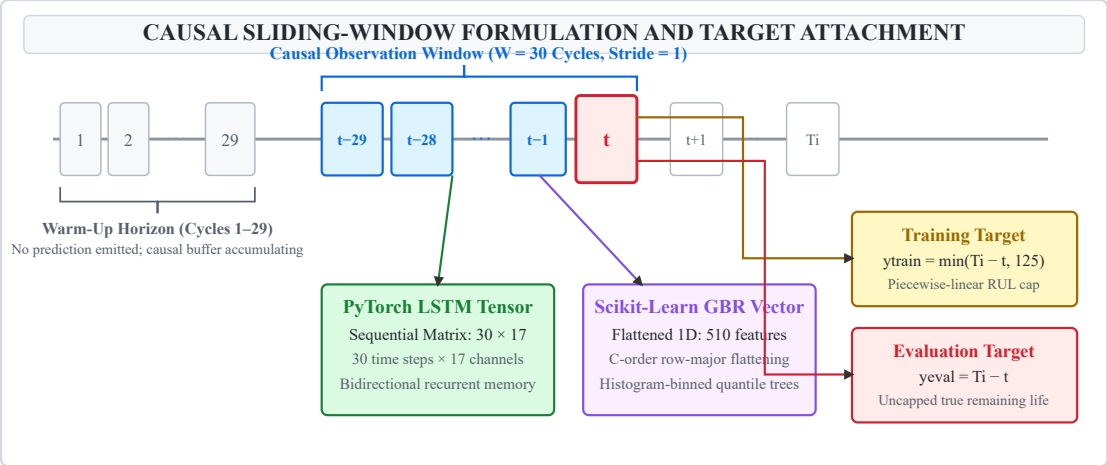


Figure 7.2: Causal Sliding Lookback Window Construction ($W = 30$), Dual Representation Mappings (Sequential LSTM Matrix vs. Flattened GBR Vector), and Target Ground Truth Assignment (Capped Training RUL vs. Uncapped Evaluation RUL).

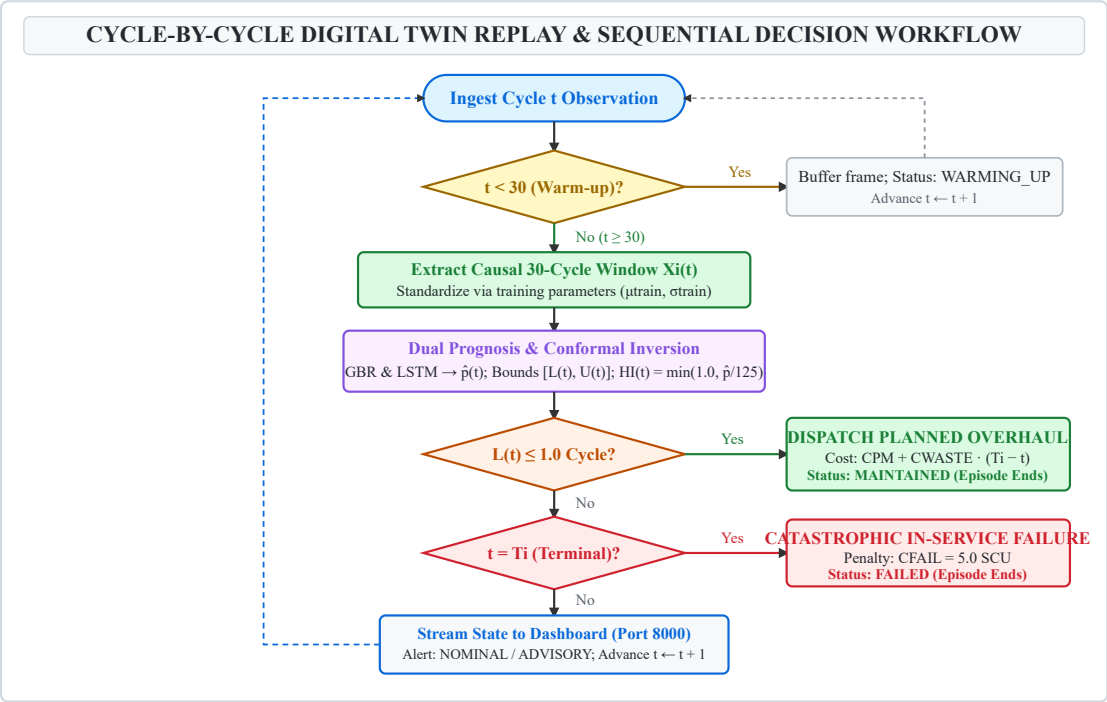


Figure 7.3: Cycle-by-Cycle Digital Twin Replay Engine and Sequential Maintenance Decision Workflow. Telemetry frames enter the causal buffer; once $t \geq 30$, point predictions and conformal bounds are computed. A maintenance intervention is dispatched when the conservative lower bound $L_i(t)$ breaches the 1.0 cycle threshold, preventing costly in-service failure while avoiding excessive wasted component life.

References

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. 2008 Int. Conf. Prognostics and Health Management (PHM)*, IEEE, 2008, pp. 1–9.

[2] Z. Huang, Y. Shen, J. Li, M. Fey, and C. Brecher, "A survey on AI-driven digital twins in Industry 4.0: Smart manufacturing and advanced robotics," *Sensors*, vol. 21, no. 19, p. 6340, 2021.

[3] R. Diao, M. Zhou, G. Meng, and S. Wang, "Turbofan engine remaining useful life prediction with reliable prediction intervals via LSTM-based quantile regression and conformal calibration," *Sensors*, vol. 26, no. 7, p. 2249, 2026.

[4] A. Javanmardi and E. Hüllermeier, "Conformal prediction intervals for remaining useful lifetime estimation," *Int. J. Progn. Health Manag.*, vol. 14, no. 2, pp. 1–14, 2023.

[5] Z. Zhu, L. Wang, M. Chen, and X. Zhao, "Predictive maintenance strategies for aero-engines considering remaining useful life prediction interval," *Qual. Reliab. Eng. Int.*, 2025, doi: 10.1002/qre.70043.

[6] C. Chen, J. Shi, N. Lu, Z. H. Zhu, and B. Jiang, "Data-driven predictive maintenance strategy considering the uncertainty in remaining useful life prediction," *Neurocomputing*, vol. 494, pp. 79–88, 2022.

[7] S. Cortes-Gomez, C. M. Patiño, Y. Byun, S. Wu, E. Horvitz, and B. Wilder, "Utility-directed conformal prediction: A decision-aware framework for actionable uncertainty quantification," in *Int. Conf. Learn. Represent. (ICLR)*, 2025.

[8] G. K. Walia and M. Kumar, "Uncertainty-aware remaining useful life prediction and PPO based optimal maintenance scheduling in industrial IoT," *Reliab. Eng. Syst. Saf.*, p. 112356, 2026.

[9] Y. Lee, R. F. Barber, and R. Willett, "Distribution-free inference with hierarchical data," *arXiv preprint arXiv:2306.06342*, 2023.

[10] C. D. Robinson, "Remaining useful life estimation for aircraft engines with risk-aware prediction intervals via conformalized quantile regression," *Int. J. Progn. Health Manag.*, vol. 17, no. 1, p. 4724, 2026.

[11] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *J. Am. Stat. Assoc.*, vol. 102, no. 477, pp. 359–378, 2007.

[12] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Soc. Psychol. Personal. Sci.*, vol. 8, no. 4, pp. 355–362, 2017.

[13] J. Wu, Y. Zhou, X. Wang, C. Chen, Y. Ma, and C. Zhang, "Research on remaining useful life prediction of equipment based on digital twins," *Sensors*, vol. 26, no. 4, p. 1240, 2026.

[14] ISO 23247-1:2021, "Automation systems and integration — Digital twin framework for manufacturing — Part 1: Overview and general principles," International Organization for Standardization, Geneva, Switzerland, 2021.